

ICT 1306

OBJECT ORIENTED PROGRAMMING

7.POLYMORPHISM

TC Irugalbandara
KHA Hettige

Polymorphism

- Polymorphism means *many* (poly) *shapes/forms* (morph).
- In the context of Object Oriented Programming, polymorphism refers to the fact :
☐ a single operation can have different behavior in different objects.
- C++ polymorphism means that a call to a member function will cause a different function to be executed depending on the type of object that invokes the function.

Signature

- In any programming language, a signature is what distinguishes one function or method from another.
- The signature includes name , number and type of parameters and also includes the *return type*
- In C, every function has to have a different name
- In C++, two methods have to differ in their *names* or in the *number* or *types* of their parameters.



In Shopping malls behave like Customer

In Bus behave like Passenger

In School behave like Student

At Home behave like Son

Type of polymorphism

- Two types
 - Compiler Time
 - Run Time
- In C++ programming you can achieve compile time polymorphism in two way,
 - Overloading
 - Two or more methods with different signatures.
 - Overriding
 - Replacing an inherited method with another having the same signature

Method Overloading

- Whenever same method name is existing multiple times in the same class with
 - different number of parameter or
 - different order of parameters or
 - different types of parametersis known as **method overloading**.

```
Class A()  
{  
  
    methodA(int a, string b);  
  
    methodA(string a,int b);  
  
} //parameters should  
  // be change
```

Write a programme by overloading “sum” function in the class Addition.

```
#include<iostream.h>
```

```
Using namespace std;
```

```
class Addition{
```

```
public:
```

```
void sum(int a, int b){
```

```
cout<<a+b; }
```

```
void sum(int a, int b, int c){
```

```
cout<<a+b+c; }
```

```
};
```

```
void main()
```

```
{
```

```
Addition obj;
```

```
obj.sum(10, 20);
```

```
cout<<endl;
```

```
obj.sum(10, 20, 30);
```

```
}
```

Method Overriding

- Define any method in both base class and derived class with same name, same parameters or signature with different implementations, this concept is known as **method overriding**.
- A derived class can have its own implementation of the base class methods.




```
#include<iostream.h>

Using namespace std;

class Base {
public:
void show() {
    cout<<"Base class"; }
};

class Derived:public Base{
public:
void show() {
    cout<<"Derived Class"; }
};
```

```
int main(){
    Base b;    //Base class object
    Derived d; //Derived class object
    b.show();  //Early Binding Occurs
    d.show();
    return 0;
}
```

Output:

Base class

Derived Class

Static Binding

```
#include <iostream>
using namespace std;

class Shape {
protected:
    int width, height;
public:
    Shape( int a=0, int b=0)
    {
        width = a;
        height = b;
    }
    int area()
    {
        cout << "Parent class area :" <<endl;
        return 0;
    }
};

class Rectangle: public Shape{
public:
    Rectangle( int a=0, int b=0):Shape(a, b) { }
    int area ()
    {
        cout << "Rectangle class area :" <<endl;
        return (width * height);
    }
};

class Triangle: public Shape{
public:
    Triangle( int a=0, int b=0):Shape(a, b) { }
    int area ()
    {
        cout << "Triangle class area :" <<endl;
        return (width * height / 2);
    }
};
```

```
int main( )
{
    Shape *shape;
    Rectangle rec(10,7);
    Triangle tri(10,5);

    // store the address of Rectangle
    shape = &rec;
    // call rectangle area.
    shape->area();

    // store the address of Triangle
    shape = &tri;
    // call triangle area.
    shape->area();

    return 0;
}
```

Output :

- Parent class area
- Parent class area

The reason for the incorrect output is :

- the call of the function area() is being set once by the compiler as the version defined in the base class.
- This is called **static resolution** of the function call, or **static linkage** - the function call is fixed before the program is executed.
- This is also sometimes called **early binding** because the area() function is set during the compilation of the program.

Example 1: Virtual Functions

- After a slight change this can be resolved.
- Insert **virtual** keyword before the base area member function.

Output:

- Rectangle class area
- Triangle class area

```
#include <iostream>
using namespace std;

class Shape {
protected:
    int width, height;
public:
    Shape( int a=0, int b=0)
    {
        width = a;
        height = b;
    }
    virtual int area()
    {
        cout << "Parent class area : " << endl;
        return 0;
    }
};

class Rectangle: public Shape{
public:
    Rectangle( int a=0, int b=0):Shape(a, b) { }
    int area ()
    {
        cout << "Rectangle class area : " << endl;
        return (width * height);
    }
};

class Triangle: public Shape{
public:
    Triangle( int a=0, int b=0):Shape(a, b) { }
    int area ()
    {
        cout << "Triangle class area : " << endl;
        return (width * height / 2);
    }
};
```

- This time, the compiler looks at the contents of the pointer instead of its type.
- Since addresses of objects of tri and rec classes are stored in *shape the respective area() function is called.
- As you can see, each of the child classes has a separate implementation for the function area(). This is how **polymorphism** is generally used.
- You have different classes with a function of the same name, and even the same parameters, but with different implementations.

Run time polymorphism (Dynamic Binding)

- In C++ Run time polymorphism can be achieved by using virtual function.
- A virtual function is a member function of a class that is declared within a base class and re-defined in a derived class.
- When you want to use the same function name in both the base and derived class, then the function in the base class is declared as virtual by using the virtual keyword and again re-defined this function in the derived class without using the virtual keyword.

```
virtual return_type function_name()
{
    .....
    .....
}
```

- Virtual function is the member function of a class that can be overridden in its derived class. It is declared with virtual keyword. Virtual function call is resolved at run-time (dynamic binding) whereas the non-virtual member functions are resolved at compile time (static binding).
- If you want to execute the member function of derived class then, you can declare display() in the base class virtual which makes that function existing in appearance only but, you can't call that function. In order to make a function virtual, you have to add keyword virtual in front of a function.

Example 2: Virtual Functions

```
#include <iostream>
using namespace std;
class B
{
    public:
        void display()
        { cout<<"Content of base class.\n"; }
};

class D : public B
{
    public:
        void display()
        { cout<<"Content of derived class.\n"; }
};

int main()
{
    B *b;
    D d;
    b->display();

    b = &d;    /* Address of object d in pointer variable */
    b->display();
    return 0;
}
```

Content of base class.
Content of base class.

```
#include <iostream>
using namespace std;
class B
{
    public:
        virtual void display()    /* Virtual function */
        { cout<<"Content of base class.\n"; }
};

class D1 : public B
{
    public:
        void display()
        { cout<<"Content of first derived class.\n"; }
};

class D2 : public B
{
    public:
        void display()
        { cout<<"Content of second derived class.\n"; }
};
```

```
int main()
{
    B *b;
    D1 d1;
    D2 d2;

    /* b->display(); // You cannot use this code here

    b = &d1;
    b->display(); /* calls display() of class derived D1 */
    b = &d2;
    b->display(); /* calls display() of class derived D2 */
    return 0;
}
```

- After the function of base class is made virtual, code `b->display( )` will call the `display( )` of the derived class depending upon the content of pointer.
- In this program, `display( )` function of two different classes are called with same code which is one of the example of polymorphism in C++ programming using virtual functions.

Example 3: Virtual Functions

```
class A
{
public:
    virtual void show()
    {
        cout<<"Hello base class";
    }
};

class B : public A
{
public:
    void show()
    {
        cout<<"Hello derive class";
    }
};
```

```
Void main(){
    A aobj;
    B bobj;
    A *bptr;
    bptr=&aobj;
    bptr→show();

    bptr=&bobj;
    bptr→show();

}
```

Output is :
Hello Base Class
Hello derived class

If virtual key word is absent ;then both
will give you
Hello Base Class

- A **virtual** function is a function in a base class that is declared using the keyword **virtual**.
- Defining in a base class a virtual function, with another version in a derived class, signals to the compiler that we don't want static linkage for this function.
- What we do want is the selection of the function to be called at any given point in the program to be based on the kind of object for which it is called.
- This sort of operation is referred to as **dynamic linkage**, or **late binding**.

Abstract class and Pure virtual Function

- Sometimes inheritance is used only for the better visualization of data and you do not need to create any object of base class.
- For example:
 - If you want to calculate area of different objects like: circle and square then, you can inherit these classes from a shape because it helps to visualize the problem but, you do not need to create any object of **shape**.
- In such case, you can declare **shape** as a abstract class.
- If you try to create object of a abstract class, compiler shows error.

Declaration of a Abstract Class

- If expression =0 is added to a virtual function then, that function becomes pure virtual function.
- Note that, adding =0 to virtual function does not assign value, it simply indicates the virtual function is a pure function.
- If a base class contains at least one virtual function then, that class is known as abstract class.
- The = 0 tells the compiler that the function has no body.


```
#include <iostream>
using namespace std;

class Shape /* Abstract class */
{
protected:
    float l;
public:
    void get_data() /* Note: this function is not virtual. */
    {
        cin>>l;
    }

    virtual float area() = 0; /* Pure virtual function */
};

class Square : public Shape
{
public:
    float area()
    { return l*l; }
};

class Circle : public Shape
{
public:
    float area()
    { return 3.14*l*l; }
};
```

```
int main()
{
    Square s;
    Circle c;
    cout<<"Enter length to calculate area of a square: ";
    s.get_data();
    cout<<"Area of square: "<<s.area();
    cout<<"\nEnter radius to calculate area of a circle:";
    c.get_data();
    cout<<"Area of circle: "<<c.area();
    return 0;
}
```

- In this program, pure virtual function virtual float area() = 0; is defined inside class Shape, so this class is an abstract class and you cannot create object of class Shape.

Thank You !